

sAIfer Lab

Joint lab on Safety and Security of AI

Strutture condizionali



UNICA

UNIVERSITÀ
DEGLI STUDI
DI CAGLIARI



sAIfer Lab

Joint lab on Safety and Security of AI

Strutture condizionali

if, if-else, if-elif-else, strutture nidificate, operatori logici

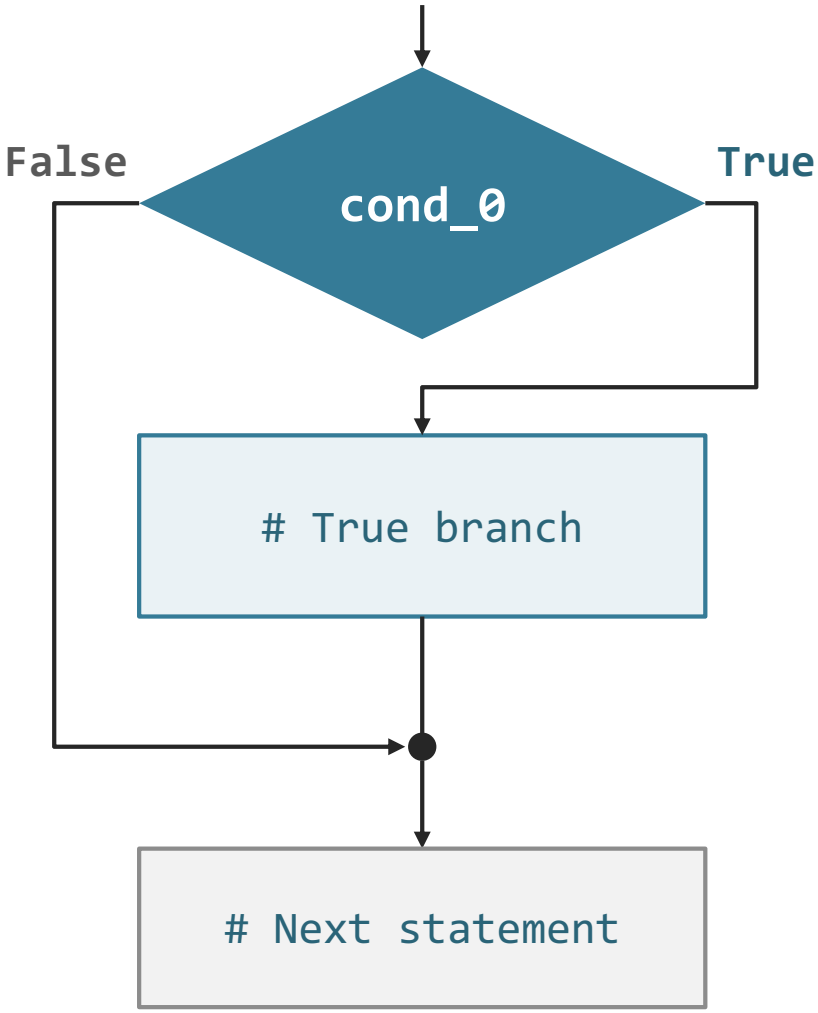
Costrutto condizionale if

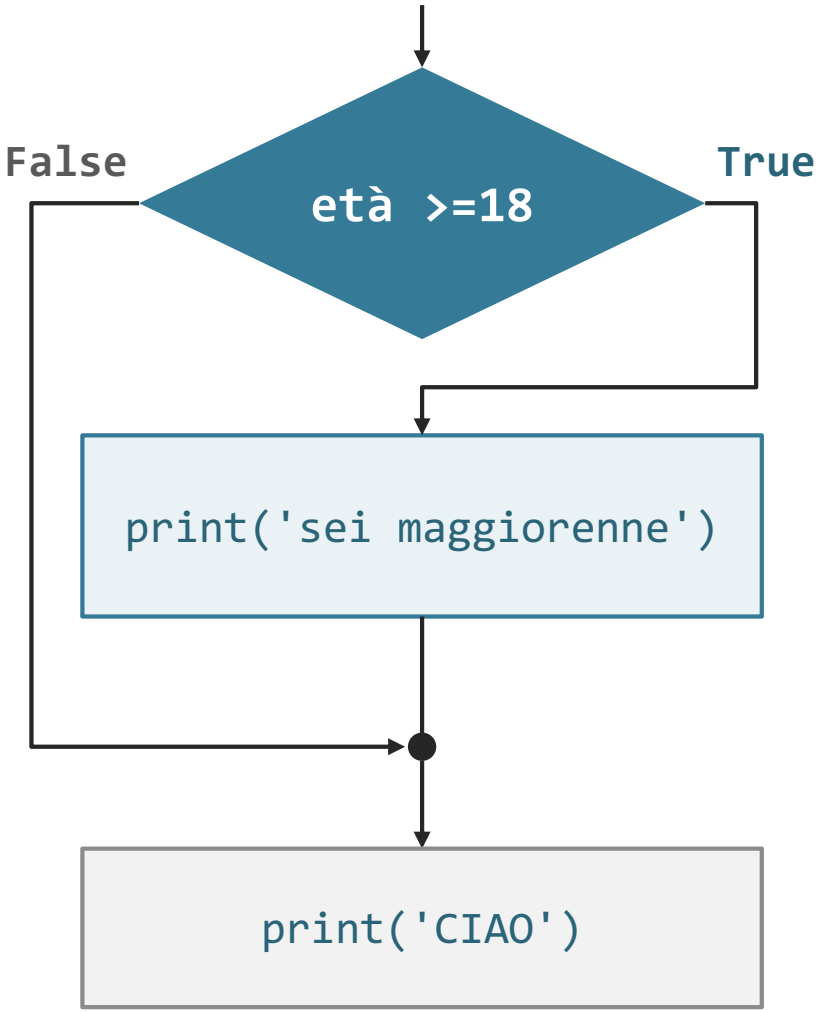
Un programma può:

- eseguire istruzioni in sequenza
- ripetere istruzioni (cicli)
- **scegliere** tra più azioni

Le strutture condizionali permettono al programma di **scegliere** quale codice eseguire in base al valore di verità di una condizione.

Diagramma di flusso di if





Sintassi minimale di if

```
if cond_0:
    # True branch
    # Run if cond_0 is true
    # Your code goes here...

# Next statement
```

- se `cond_0` è `True` viene eseguito il blocco associato (ramo `True`)
- se `cond_0` è `False` il blocco viene saltato
- In entrambi i casi si prosegue con `# Next statement`

Blocco di codice

- Un **blocco di codice** è un insieme di istruzioni che appartengono a una struttura di controllo (funzione, ciclo, `if`)
- Il blocco è **indentato**: spostato a destra rispetto all'istruzione che lo controlla
- In Python l'indentazione è **obbligatoria**: stabilisce quali istruzioni appartengono al blocco

Esempio: maggiorenne o minorenne?

```
print("quanti anni hai?")
eta = int(input())
if eta >= 18:
    print("sei maggiorenne")
```

Esempio con errore: questo codice **non** stampa “sei maggiorenne” se hai esattamente 18 anni.

```
print("quanti anni hai?")
eta = int(input())
if eta > 18:
    print("sei maggiorenne")
```

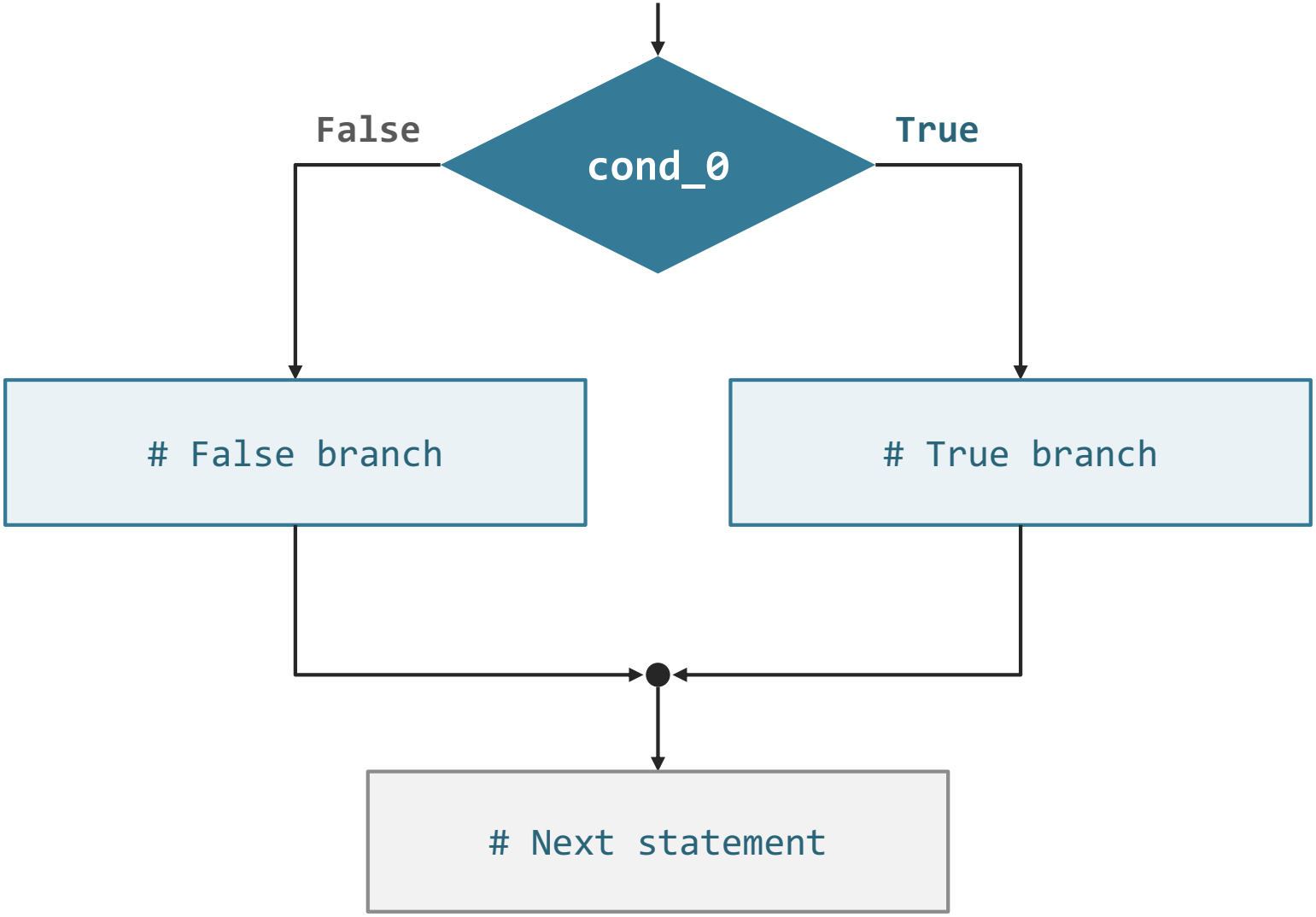
Regole sintattiche del costrutto if

IMPORTANTE

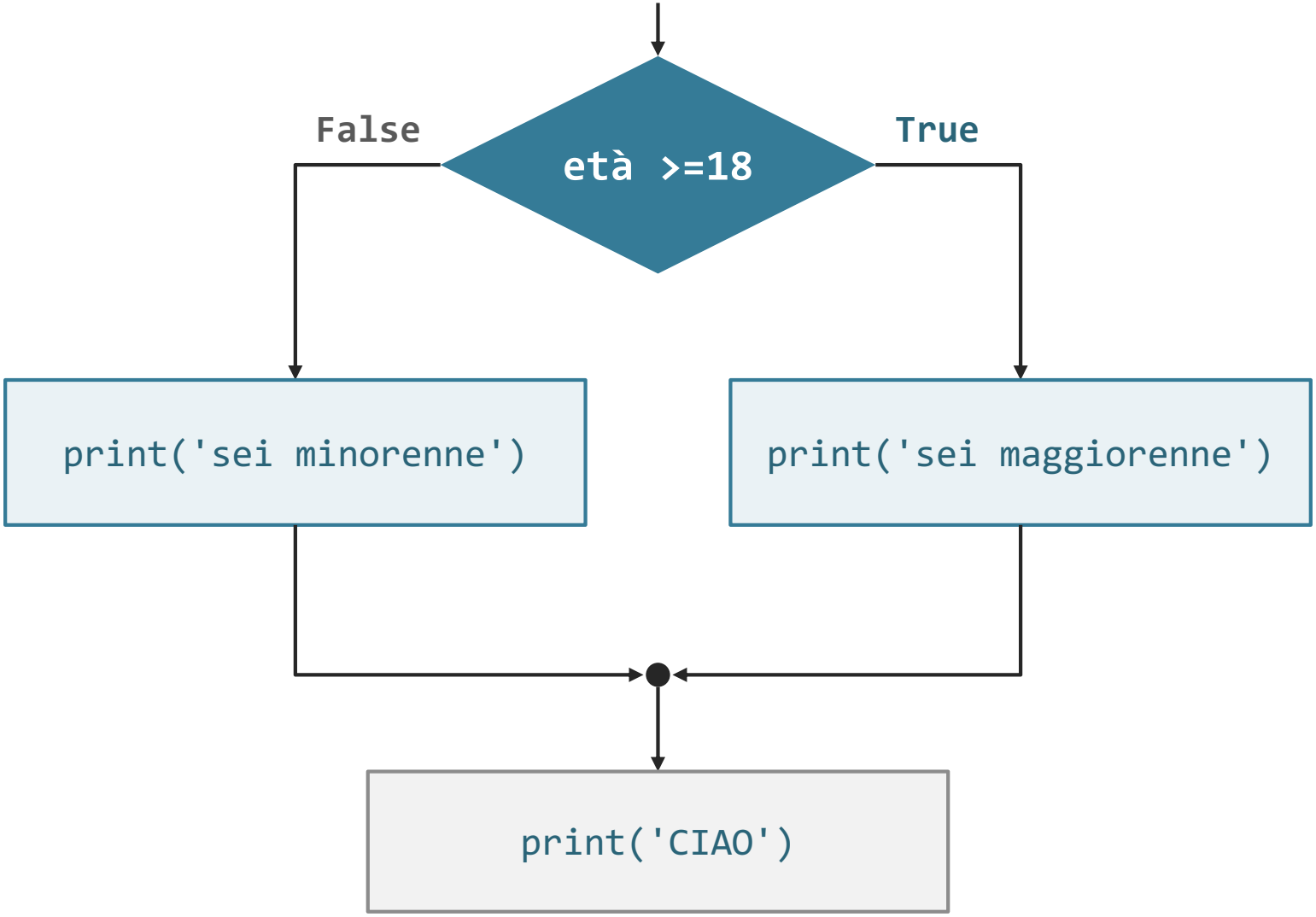
- Dopo la condizione i **due punti** `:` sono obbligatori
- Le istruzioni del blocco vanno **indentate**, tutte con lo stesso numero di spazi (per convenzione **4 spazi**)
- Il blocco termina alla prima istruzione che torna al livello di indentazione dell'`if`

Se una regola sintattica non è rispettata, Python non esegue il programma e segnala un errore.

IF -ELSE



IF -ELSE


età >=18
False
True
print('sei minorenne')
print('sei maggiorenne')
print('CIAO')

IF - ELSE

A seconda del valore della condizione viene eseguito il ramo `True` oppure il ramo `False`.

```
if cond_0:
    # True branch
    # Run if cond_0 is true
    # Your code goes here...
else:
    # False branch
    # Run if cond_0 is false
    # Your code goes here...

# Next statement
```

Esempio: maggiorenne o minorenne?

```
print("quanti anni hai?")
eta = int(input())
if eta >= 18:
    print("sei maggiorenne")
else:
    print("sei minorenne")
```

Un ramo True vuoto: pass

- Il ramo `True` è **obbligatorio**, anche se voglio eseguire codice solo quando la condizione è falsa
- `pass` è l'istruzione vuota ("nessuna azione")

```
if cond:
    pass
else:
    print("sono nel ramo False")
```

docs.python.org/it/3.12/reference/simple_stmts.html#the-pass-statement

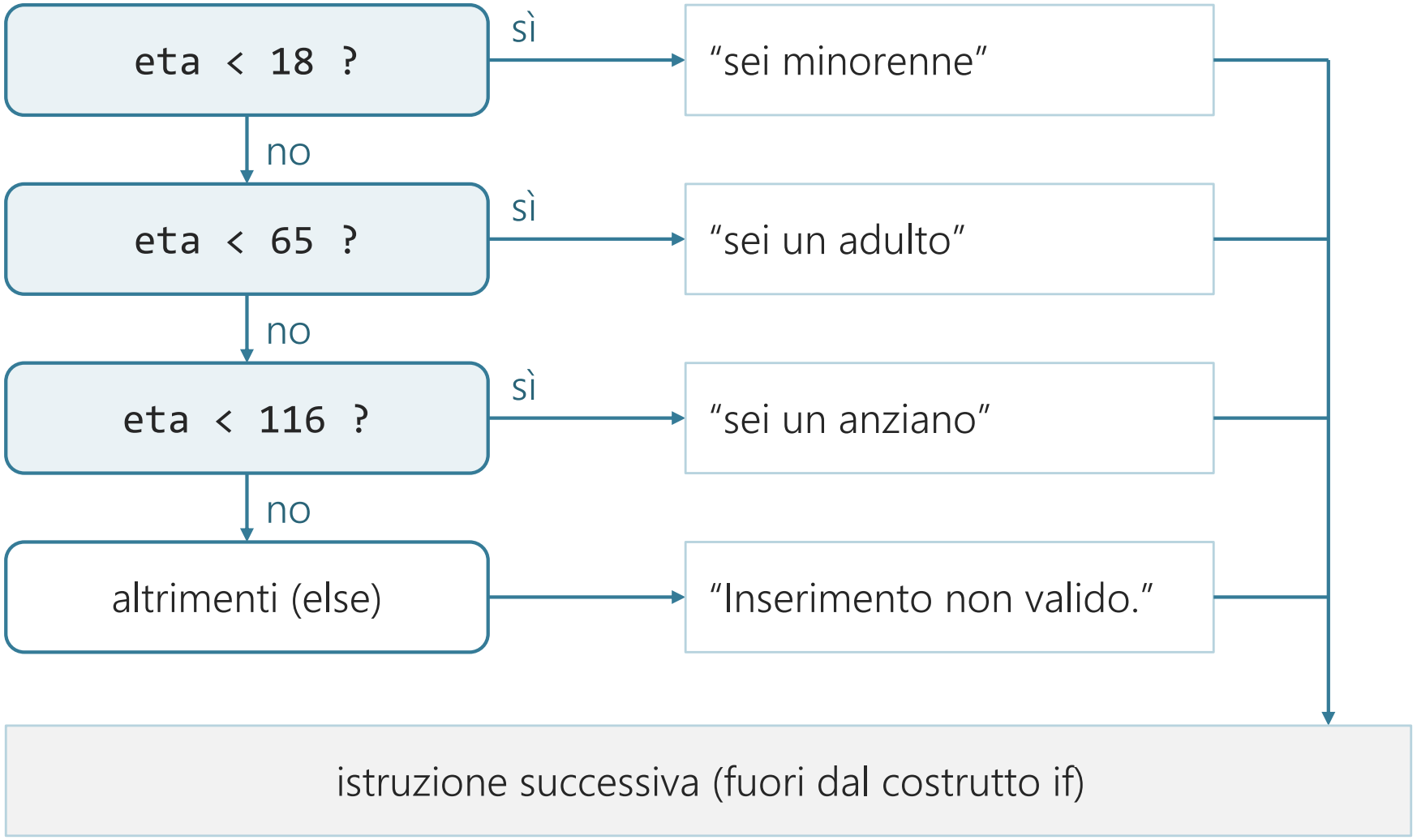
Negare la condizione

Per eseguire codice solo quando la condizione è falsa, il modo naturale è **negarla**.

```
if not cond:
    print("sono nel ramo True, la condizione è falsa")
```

`pass` resta utile come **segnaposto**, per esempio per codice ancora da scrivere.

IF - ELIF - ELSE


eta < 18 ?
sì
“sei minorenne”
no
eta < 65 ?
sì
“sei un adulto”
no
eta < 116 ?
sì
“sei un anziano”
no
altrimenti (else)
“Inserimento non valido.”
istruzione successiva (fuori dal costrutto if)

IF - ELIF - ELSE

Più condizioni da testare “in cascata”:

- Valuto `condizione_1`.
  Se è vera, eseguo `azione_1` e **termino**
- **Solo se** la `condizione_1` **è falsa**, valuto `condizione_2`.
  Se è vera, eseguo `azione_2` e **termino**
- **Solo se** la `condizione_2` **è falsa**, valuto `condizione_3`.
  Se è vera, eseguo `azione_3` e **termino**
- …
- Se tutte le condizioni sono false,
  eseguo `azione_finale` e **termino**

IF - ELIF - ELSE: il codice

```
print("quanti anni hai?")
eta = int(input())

if eta < 18:
    print("sei minorenne")
elif eta < 65:
    print("sei un adulto")
elif eta < 116:
    print("sei un anziano")
else: # OPZIONALE!
    print("Inserimento non valido o età fuori dal range.")
```

- Se arrivo a un `elif`, le condizioni precedenti sono false
- Se arrivo a `else`, tutte le condizioni sono false
- In ogni caso viene eseguito (al più) un solo blocco di codice

Sintassi completa del costrutto if

Obbligatori solo la parola chiave `if`, la prima condizione e il blocco `True`. Tutto il resto è opzionale.

```
# solo IF
if cond_0:
    # CODICE DA ESEGUIRE...
# Next statement
```

Sintassi completa del costrutto if

```
# IF ELSE
if cond_0:
    # CODICE DA ESEGUIRE...
else:
    # CODICE ALTERNATIVO DA ESEGUIRE...
# Next statement
```

Sintassi completa: IF - ELIF - ELSE

```
if cond_0:
    # Valuta cond_0.
    # Se cond_0 è True, esegui il codice ed esci dall'IF
elif cond_1:
    # Solo se cond_0 è False, valuta cond_1.
    # Se cond_1 è True, esegui il codice ed esci dall'IF
elif cond_2:
    # Solo se cond_1 è False, valuta cond_2.
    # Se cond_2 è True, esegui il codice ed esci dall'IF
...
else:
    # Solo se tutte le condizioni precedenti sono False,
    # esegui il codice ed esci dall'IF
# Next statement
```

Esempi

`if`
solo ramo vero

Se piove, prendo l'ombrello.

Se non piove, non succede nulla: si prosegue.

`if-else`
due alternative

Se la password è corretta, accedo al sito; altrimenti vedo un messaggio di errore.

Uno dei due rami viene sempre eseguito.

`if-elif-else`
più alternative

Se fa meno di 10 °C, metto il cappotto; altrimenti, se fa meno di 20 °C, metto una giacca; altrimenti esco in maglietta.

Le condizioni si valutano in ordine: vale la prima vera.



sAIfer Lab

Joint lab on Safety and Security of AI

Strutture nidificate

Strutture nidificate (nested if-elif-else)

- In ogni ramo (`if`, `elif`, `else`) può comparire qualsiasi istruzione Python, compreso **un altro costrutto condizionale**
- L'`if` interno viene valutato solo se `cond_0` è vera
- Ogni `elif` o `else` appartiene all'`if` che lo precede **allo stesso livello di indentazione**
- Non ci sono limiti alla profondità, ma strutture troppo complesse diventano difficili da capire e modificare

Esempio di struttura nidificata

Se oggi è un giorno festivo:

> **Se** c'è il sole, vado al mare;
> **altrimenti** vado al cinema.

Altrimenti (giorno feriale), vado a scuola.

La domanda sul sole si pone **solo se** la prima condizione è vera: la seconda decisione è nidificata nel primo ramo.

Esercizio: una soluzione più robusta

Se si controlla solo `s == "feriale"`, ogni altro input va nel ramo `else`.

```
s = input("Feriale o festivo? ")
if s == "feriale":
    print("Vado a scuola")
else:
    sole = input("C'è il sole? ")
    if sole == "si":
        print("Vado al mare")
    else:
        print("Vado al cinema")
```

Rendila **più robusta**: ignora maiuscole e minuscole e **segnala un errore** per ogni input non valido, sul giorno e sul sole.

Suggerimenti

- trasformare tutto in maiuscole (`.upper()`)
- controllare sia festivo sia feriale; ogni altro valore è un errore
- stesso controllo per la risposta sul sole (sì/no)

Strutture nidificate (nested if-elif-else)

```
if cond_0:
    if cond_01: # cond_10 valuta solo se cond_0 è True
        #    CODICE DA ESEGUIRE...
    elif cond_11:
        #    CODICE DA ESEGUIRE...
    else:
        #    CODICE DA ESEGUIRE...
elif cond_1:
    #    CODICE DA ESEGUIRE...
elif cond_2:
    #    CODICE DA ESEGUIRE...
else:
    #    CODICE DA ESEGUIRE...
# Next statement
```



sAIfer Lab

Joint lab on Safety and Security of AI

Condizioni

Strutture condizionali e operatori logici

La condizione di un `if` può combinare più condizioni con gli operatori logici `and`, `or`, `not`.



Arrampicata

più di 14 anni oppure più alto di 140 cm



Borsa di studio

media sopra 28 e al massimo 24 anni



Sconto

ai soci, oppure agli studenti sotto i 26 anni



Gita in bici

niente pioggia né vento, ed è weekend

Condizioni ben strutturate evitano `if` nidificati non necessari e il codice resta leggibile.

Strutture condizionali e operatori logici

La condizione di un `if` può combinare più condizioni con gli operatori logici `and`, `or`, `not`.

```
eta > 14 or altezza > 140
# arrampicata: più di 14 anni oppure più alto di 140 cm

media > 28 and eta <= 24
# borsa di studio: media sopra 28 e al massimo 24 anni

socio or (eta < 26 and studente)
# sconto: ai soci, oppure agli studenti sotto i 26 anni

not (pioggia or vento) and (sabato or domenica)
# gita in bici: niente pioggia né vento, ed è weekend
```

Condizioni ben strutturate evitano `if` nidificati non necessari e il codice resta leggibile.

Variabili booleane nelle condizioni

Una variabile può contenere direttamente un **valore booleano**: `True` o `False`.

```
eta = int(input('Quanti anni hai? '))
maggiorenne = eta >= 18
```

Confronto esplicito

```
if maggiorenne == True:
    print('Puoi votare')
```

Forma equivalente, più leggibile

```
if maggiorenne:
    print('Puoi votare')
```

`maggiorenne` vale già `True` o `False`: il confronto `== True` non serve.

Confronti concatenati

Per verificare se un valore è in un intervallo, Python permette confronti **concatenati**, come in matematica:

```
# equivale a: 18 <= eta and eta < 65
if 18 <= eta < 65:
    print("sei un adulto")
```

Utili quando gli intervalli vanno scritti in modo esplicito, **senza dipendere dall'ordine dei rami**.

Confronti concatenati: esempio

```
if 0 <= eta < 18:
    print("sei minorenne")
elif 18 <= eta < 65:
    print("sei un adulto")
elif 65 <= eta < 116:
    print("sei un anziano")
else:
    print("Inserimento non valido o età fuori dal range.")
```

- Ora anche un'età negativa viene segnalata come non valida
- Usare solo la forma ordinata `a < x < b`



sAIfer Lab

Joint lab on Safety and Security of AI

Errori comuni

Errori di sintassi nel costrutto if

IMPORTANTE

```
if eta >= 18                    # mancano i due punti
    print("sei maggiorenne")
# SyntaxError: expected ':'
```

```
if eta >= 18:
print("sei maggiorenne")      # non indentata
# IndentationError: expected an indented block
#                   after 'if' statement
```

```
if eta >= 18:
    print("sei maggiorenne")
  print("puoi votare")        # 2 spazi invece di 4
# IndentationError: unindent does not match
#                   any outer indentation level
```

Errore tipico: = invece di ==

IMPORTANTE

Nella condizione si usa l'operatore di confronto ==, non l'assegnazione =.

```
if x = 5:       # ERRATO: = assegna, non confronta
    print("x vale 5")
# SyntaxError: invalid syntax.
# Maybe you meant '==' or ':=' instead of '='?
```

```
if x == 5:      # CORRETTO: confronta x con 5
    print("x vale 5")
```



sAIfer Lab

Joint lab on Safety and Security of AI

Altre considerazioni

If separati: tutte le condizioni valutate

- Gli `if` non sono collegati da `elif`: **tutte le condizioni** vengono sempre valutate, ed è ciò che serve
- La scelta multipla si può scrivere con `if` indipendenti, ma la soluzione è subottima (perché?)

```
if op == "+":
    r = a + b
if op == "-":
    r = a - b
if op == "*":
    r = a * b
```

In questa semplice 'calcolatrice' **op** è l'operazione da effettuare

Con elif: si valuta solo fino alla prima vera

- Con `elif` le condizioni sono collegate: appena una è vera, **le altre vengono saltate**
- **Nessun confronto inutile**: è la forma corretta per la scelta multipla

```
if op == "+":
    r = a + b
elif op == "-":   # valutata solo se op non è "+"
    r = a - b
elif op == "*":   # valutata solo se op non è "+" né "-"
    r = a * b
```

Divisori di n: servono if separati

- Quali fra 2, 3, 5, 7 dividono n? Le risposte **non si escludono**: 30 è divisibile per 2, 3 e 5
- Con `elif` troveremmo solo il primo divisore: **ogni condizione va valutata**

```
n = int(input("Inserisci un numero: "))

if n % 2 == 0:
    print("2 è divisore di", n)
if n % 3 == 0:
    print("3 è divisore di", n)
if n % 5 == 0:
    print("5 è divisore di", n)
if n % 7 == 0:
    print("7 è divisore di", n)
```

Un else per tutti gli if?

- Vogliamo segnalare il caso in cui il numero `a` non è divisibile per **nessuno** dei divisori elencati
- esempio: 13 non è divisibile per 2, 3, 5, 7
- Servirebbe un `else` collegato a tutti gli `if`: in Python non è possibile
- `else` si associa solo all'`if` (o `elif`) immediatamente precedente, allo stesso livello di indentazione

Divisori di a: servono if separati

- Quali fra 2, 3, 5, 7 dividono 13?

```
a = 13

if a % 2 == 0:
    print("2 è divisore di", a)
if a % 3 == 0:
    print("3 è divisore di", a)
if a % 5 == 0:
    print("5 è divisore di", a)
if a % 7 == 0:
    print("7 è divisore di", a)
else:
    print("nessun divisore")
```

Vorremmo che else venisse eseguito solo se TUTTE le condizioni sono false.

In Python non è possibile: else si lega solo all'ultimo if.

Con a = 10 l'else stampa un risultato sbagliato

- Proviamo con a = 10: che cosa stampa il programma?

```
a = 10

if a % 2 == 0:
    print("2 è divisore di", a)
if a % 3 == 0:
    print("3 è divisore di", a)
if a % 5 == 0:
    print("5 è divisore di", a)
if a % 7 == 0:
    print("7 è divisore di", a)
else:
    print("nessun divisore")
```

Output

```
2 è divisore di 10
5 è divisore di 10
nessun divisore
```

10 % 7 ≠ 0, quindi l'else dell'ultimo if viene eseguito, anche se 2 e 5 sono divisori.

Soluzione: variabile flag

Il flag `divisibile` vale `False` e diventa `True` se almeno una condizione è vera.

```
divisibile = False # flag: diventa True se a è divisibile
a = int(input())
if a % 2 == 0:
    divisibile = True
    print('divisibile per 2')
if a % 3 == 0:
    divisibile = True
    print('divisibile per 3')
if a % 5 == 0:
    divisibile = True
    print('divisibile per 5')
#-----
if not divisibile:
    print('Non è divisibile per 2, 3 o 5')
```

Variabile contatore

E se volessimo sapere **quanti** sono i divisori di a?

- Una variabile **contatore** conta quante volte si verifica una condizione
- Si inizializza a 0 e si incrementa di 1 a ogni evento
- Esempio: a è divisibile per 2, 3 o 5? E per quanti di questi divisori?

```
n_divisori = 0 # contatore: parte da 0
a = int(input())
if a % 2 == 0:
    n_divisori = n_divisori + 1 # +1 a ogni divisore
```

Variabile contatore: il codice

```
a = int(input())
n_divisori = 0 # VARIABILE CONTATORE
if a % 2 == 0:
    print('divisibile per 2')
    n_divisori = n_divisori + 1
if a % 3 == 0:
    print('divisibile per 3')
    n_divisori = n_divisori + 1
if a % 5 == 0:
    print('divisibile per 5')
    n_divisori = n_divisori + 1
if n_divisori == 0:
    print('il numero', a,
          'non è divisibile per i divisori esaminati')
else:
    print('il numero', a, 'è divisibile per',
          n_divisori, 'dei divisori esaminati')
```

Variabile contatore: osservazioni

- Ogni condizione vera incrementa il contatore di 1
- I tre `if` sono **indipendenti**: un numero può essere divisibile per più divisori

La leggibilità del codice

Un problema può avere diverse soluzioni, tutte corrette.

Un programma deve essere comprensibile alle persone, non solo al computer.



Comprensione

la logica del programma è chiara a chi legge



Debug

gli errori si trovano più facilmente



Modifiche

meno rischi quando il codice viene modificato o esteso

Tra soluzioni equivalenti, preferire quella che rende **più chiara la logica**, anche con qualche riga in più.



sAIfer Lab

Joint lab on Safety and Security of AI

Esempi ed esercizi

Accessi, prenotazioni, prestiti, ordini, voti

Esempio: accesso a un'area riservata

Un utente può accedere solo se:

- è loggato
- ha il ruolo di admin **oppure** è un utente premium
- il suo account è attivo

Nella versione nidificata l'indentazione stabilisce a quale `if` si riferiscono gli `elif` e gli `else`.

Area riservata: versione nidificata

```
utente_loggato = True
ruolo_admin = False
utente_premium = True
account_attivo = True

if utente_loggato:
    if account_attivo:
        if ruolo_admin:
            print("Accesso consentito - sei admin")
        elif utente_premium:
            print("Accesso consentito - sei premium")
        else:
            print("Accesso negato: permessi insufficienti")
    else:
        print("Accesso negato: account non attivo")
else:
    print("Accesso negato: utente non loggato")
```

Area riservata: condizioni booleane

La logica si comprime in **un'unica espressione booleana**: se è vera l'accesso è consentito, altrimenti è negato.

```
utente_loggato = True
ruolo_admin = False
utente_premium = True
account_attivo = True

if (utente_loggato and (ruolo_admin or utente_premium)
        and account_attivo):
    print("Accesso consentito")
else:
    print("Accesso negato")
```

Nidificata o flat?

Nidificata quando:

- servono messaggi di errore specifici
- le condizioni dipendono l'una dall'altra
- un controllo ha senso solo se i precedenti sono superati

Flat quando:

- si vuole codice compatto
- le condizioni stanno in un'unica espressione
- non serve sapere **perché** la condizione è falsa

L'approccio migliore dipende dal contesto e dalle esigenze di chiarezza e manutenzione.

Un mix fra i due approcci

```
if utente_loggato:
    if account_attivo:
        if ruolo_admin or utente_premium:
            print("Accesso consentito")
        else:
            print("Accesso negato: permessi insufficienti")
    else:
        print("Accesso negato: account non attivo")
else:
    print("Accesso negato: utente non loggato")
```

Esercizi (1/3)

- Chiedere all'utente 3 numeri **diversi** e dire qual è il più grande e qual è il più piccolo
- Chiedere 3 numeri, dire qual è il più grande e il più piccolo, e se sono tutti diversi, tutti uguali o due uguali

Esercizi (2/3): la patente

- Chiedere età e crimine commesso (0: nessuno; da 1 a 10: via via più gravi; al massimo uno)
- Decidere se l'utente può conseguire la patente: definire l'età minima e il livello di crimine oltre il quale non viene rilasciata
- Variante: due tipi di patente (es. 16 e 18 anni)

Oltre un certo grado di complessità (`if` di `if` di `if`…) servono altri strumenti, che vedremo in seguito.

Esercizi (3/3)

- Dati 3 numeri, verificare se possono essere i lati di un triangolo; in caso affermativo dire se è isoscele, equilatero o scaleno
- Chiedere i coefficienti di un'equazione di secondo grado $a\ x^2 + b\ x + c = 0$ e trovarne le radici, anche complesse

Sistema di prenotazione biglietti

Un cliente può acquistare un biglietto solo se:

- ha almeno 18 anni, **oppure** ha almeno 14 anni (ma meno di 18) ed è accompagnato da un adulto
- ha abbastanza saldo nel conto per pagare il biglietto
- ci sono ancora posti disponibili

Prenotazione: bozza con if nidificati

```
limite_eta_1 = 14
limite_eta_2 = 18

# Controllo età e accompagnamento
if (eta >= limite_eta_2
        or (eta >= limite_eta_1 and accompagnato)):
    if saldo >= prezzo_biglietto:  # Controllo saldo
        # Controllo disponibilità posti
        if posti_disponibili > 0:
            print("Prenotazione confermata!")
        else:
            print("Prenotazione negata: posti esauriti")
    else:
        print("Prenotazione negata: saldo insufficiente")
else:
    print("Prenotazione negata: età non idonea")
```

Prenotazione: condizioni booleane

```
if ((eta >= limite_eta_2
        or (eta >= limite_eta_1 and accompagnato))
        and saldo >= prezzo_biglietto
        and posti_disponibili > 0):
    print("Prenotazione confermata!")
else:
    print("Prenotazione negata")
```

Prenotazione: condizioni con un nome

Ancora meglio: dare un nome a ogni condizione.

```
condizione_eta_accompagnamento = (
    eta >= limite_eta_2
    or (eta >= limite_eta_1 and accompagnato))
condizione_economica = (saldo >= prezzo_biglietto)
condizione_posti = posti_disponibili > 0

if (condizione_eta_accompagnamento
        and condizione_economica and condizione_posti):
    print("Prenotazione confermata!")
else:
    print("Prenotazione negata")
```

Prestito bancario

Un software bancario decide se approvare un prestito:

- il cliente deve avere almeno 18 anni
- almeno 700 punti di credito: approvato subito
- tra 600 e 699 punti: approvato se il reddito mensile supera 3000 €, altrimenti rifiutato
- meno di 600 punti: rifiutato

Gestione ordini

Tempi di consegna stimati in base alla destinazione e al tipo di spedizione.

Destinazione	Standard	Express
Italia	5 giorni	2 giorni
Europa	7 giorni	3 giorni
Resto del mondo	15 giorni	7 giorni

- La bozza di soluzione usa le variabili flag `errore_spedizione` e `errore_destinazione`

Gestione ordini: bozza (1/2)

```
destinazione = "Europa"
spedizione = "Express"
errore_spedizione = False
errore_destinazione = False
if destinazione == "Italia":
    if spedizione == "Standard":
        giorni = 5
    elif spedizione == "Express":
        giorni = 2
    else:
        errore_spedizione = True  # Opzione non valida
elif destinazione == "Europa":
    if spedizione == "Standard":
        giorni = 7
    elif spedizione == "Express":
        giorni = 3
    else:
        errore_spedizione = True
```

Gestione ordini: bozza (2/2)

```
elif destinazione == "Resto del mondo":
    if spedizione == "Standard":
        giorni = 15
    elif spedizione == "Express":
        giorni = 7
    else:
        errore_spedizione = True
else:
    # Destinazione sconosciuta
    errore_destinazione = True

if not (errore_spedizione or errore_destinazione):
    print("Tempi di consegna stimati:", giorni, "giorni.")
else:
    print("Errore: destinazione o tipo di spedizione",
          "non valido.")
```

Gestione ordini: da provare

- Riformulare la soluzione con flag di significato opposto, es. `spedizione_corretta`
- Riformulare la soluzione con altre forme della struttura condizionale

Esempio: voto in base al punteggio

- almeno 90: “Ottimo”; altrimenti, almeno 75: “Buono”
- altrimenti, almeno 60: “Sufficiente”
- in tutti gli altri casi: “Insufficiente”

```
punteggio = int(input())
if punteggio >= 90:
    print("Ottimo")
elif punteggio >= 75:
    print("Buono")
elif punteggio >= 60:
    print("Sufficiente")
else:
    print("Insufficiente")
```

Esempio: promosso, rimandato o bocciato

Valuta un voto e classifica l'esito: promosso con lode, promosso, rimandato, bocciato.

```
print("Inserisci il voto (0-100):")
voto = int(input())

if voto >= 60:
    if voto >= 90:
        print("Promosso con lode!")
    else:
        print("Promosso")
elif voto >= 40:
    print("Rimandato")
else:
    print("Bocciato")
```